

Evidencia grafica de práctica.

- Preparación de entorno.

```
D:\>mkdir NNA

D:\>cd NNA

D:\NNA>pu -3.11 -m venv .venv
"pu" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

D:\NNA>py -3.11 -m venv .venv

D:\NNA>
D:\NNA>.venv\Scripts\activate

(.venv) D:\NNA>python --version
Python 3.11.9

(.venv) D:\NNA>set TEMP=D:\temp

(.venv) D:\NNA>set TEMP=D:\temp

(.venv) D:\NNA>echo %TEMP%
D:\temp

(.venv) D:\NNA>pip install tensorflow numpy matplotlib pillow
Collecting tensorflow
  Using cached tensorflow-2.21.0-cp311-cp311-win_amd64.whl.metadata (4.5 k
B)
```

```
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) D:\NNA>python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in d:\nna\.venv\lib\site-packages (24.0
)
Collecting pip
  Using cached pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
  Using cached pip-26.2.1-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
  Successfully installed pip-26.2.1
```

```
(.env) D:\NNA>python -c "import tensorflow as tf; import numpy as np; import matplotlib; from PIL import Image; print('TensorFlow:', tf.__version__); print('Dependencias listas')"
```

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1787799315.131561 30780 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1787799329.902264 30780 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.

TensorFlow: 2.21.0

Dependencias listas

```
benchmark_networks.py • test_my_handwriting.py •
benchmark_networks.py > ...
58 |         if CNN in name:
59 |             x_train_input = np.expand_dims(x_train, -1)
60 |             x_test_input = np.expand_dims(x_test, -1)
61 |
62 |             model.compile(optimizer='adam',
63 |                           loss='sparse_categorical_crossentropy',
64 |                           metrics=['accuracy'])
65 |
66 |             history = model.fit(x_train_input, y_train, epochs=epochs,
67 |                                validation_data=(x_test_input, y_test))
68 |
69 |             loss, acc = model.evaluate(x_test_input, y_test)
70 |             print(f"Test Accuracy ({name}): {acc*100:.2f}%")
71 |             histories[name] = history.history['val_accuracy']
72 |
73 |     return histories
```

- Resultados obtenidos.

```

--- Training: MLP (Dense) ---
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
844/844 ██████████ 11s 4ms/step - accuracy: 0.8986 - loss: 0.3506 - val_accuracy: 0.9593 - val_loss: 0.1471
Epoch 2/3
844/844 ██████████ 3s 3ms/step - accuracy: 0.9500 - loss: 0.1720 - val_accuracy: 0.9697 - val_loss: 0.1067
Epoch 3/3
844/844 ██████████ 2s 3ms/step - accuracy: 0.9616 - loss: 0.1296 - val_accuracy: 0.9742 - val_loss: 0.0882
Test Accuracy (MLP (Dense)): 96.93%

```

```

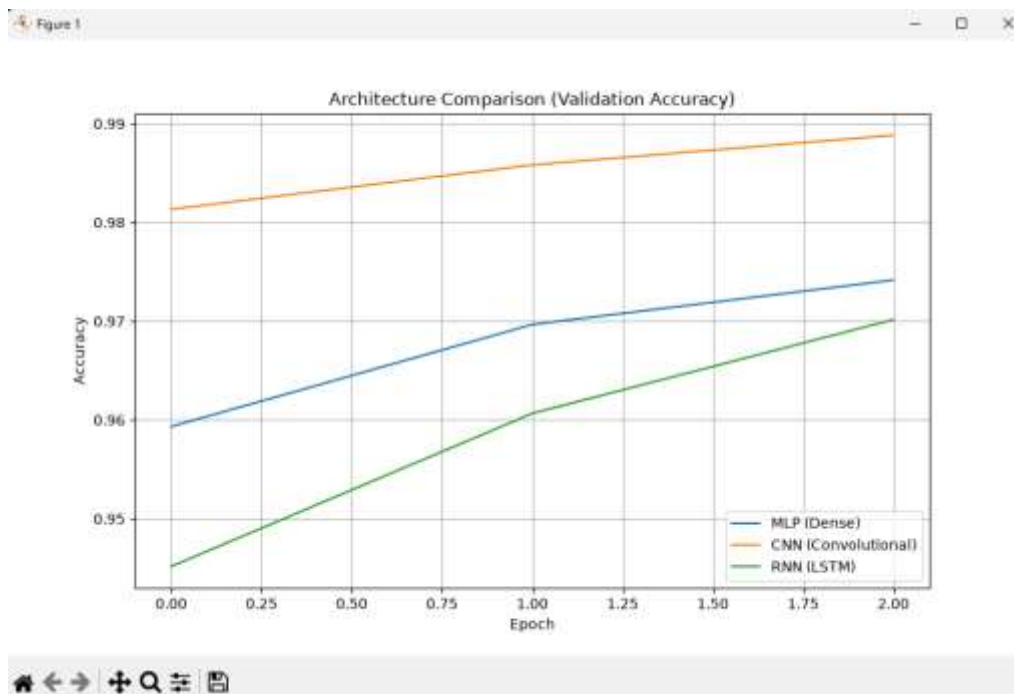
--- Training: RNN (LSTM) ---
Epoch 1/3
844/844 ██████████ 11s 11ms/step - accuracy: 0.8240 - loss: 0.5506 - val_accuracy: 0.9452 - val_loss: 0.1873
Epoch 2/3
844/844 ██████████ 8s 10ms/step - accuracy: 0.9495 - loss: 0.1696 - val_accuracy: 0.9607 - val_loss: 0.1246
Epoch 3/3
844/844 ██████████ 9s 11ms/step - accuracy: 0.9647 - loss: 0.1169 - val_accuracy: 0.9702 - val_loss: 0.0991
Test Accuracy (RNN (LSTM)): 96.74%

```

```

--- Training: CNN (Convolutional) ---
Epoch 1/3
844/844 ██████████ 16s 16ms/step - accuracy: 0.9526 - loss: 0.1510 - val_accuracy: 0.9813 - val_loss: 0.0607
Epoch 2/3
844/844 ██████████ 20s 16ms/step - accuracy: 0.9849 - loss: 0.0476 - val_accuracy: 0.9858 - val_loss: 0.0487
Epoch 3/3
844/844 ██████████ 20s 16ms/step - accuracy: 0.9899 - loss: 0.0313 - val_accuracy: 0.9888 - val_loss: 0.0392
Test Accuracy (CNN (Convolutional)): 98.87%

```



- Tabla con resultados – Parte 1.

Evaluated Model	Original Accuracy (%)	Improved Accuracy (Regularization - Part 2)	Successfully Identified Tricky Drawing? (Yes / No)
MLP (Dense Fully-Connected)	96.93%	[Insert new % here]	[Insert Yes / No]
CNN (Spatial Convolutional)	98.87%	Not Applicable (Baseline)	[Insert Yes / No]
RNN (Sequential LSTM)	96.74%	Not Applicable (Baseline)	[Insert Yes / No]

- Consulta a ChatGPT.

“¿Por qué una red MLP suele presentar sobreajuste cuando trabaja con imágenes y qué técnica de regularización puedo utilizar en Keras para evitarlo?”

Una red MLP suele presentar sobreajuste con imágenes porque primero aplanar la imagen en un vector y pierde la relación espacial entre los píxeles. Además, cada neurona de una capa densa se conecta con muchas entradas, lo que genera una gran cantidad de parámetros. Con tantos pesos disponibles, el modelo puede terminar memorizando detalles y ruido del conjunto de entrenamiento en lugar de aprender patrones que funcionen bien con imágenes nuevas.

Una técnica muy útil en Keras para reducir este problema es **Dropout**. Durante el entrenamiento, Dropout desactiva aleatoriamente una proporción de neuronas en cada lote. Por ejemplo, `Dropout(0.3)` desactiva aproximadamente el 30 % de las activaciones temporalmente.

Esto obliga a la red a no depender demasiado de neuronas específicas y favorece que aprenda representaciones más generales. En términos prácticos, puede reducir la diferencia entre la precisión de entrenamiento y la de prueba.

- Implementación de Dropout.

```
def build_mlp():
    model = models.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(10, activation='softmax')
    ])
    return model, "MLP (Dense)"
```

- Solicitud a ChatGPT.

devuelve únicamente el fragmento de código de Keras necesario para integrar la técnica de regularización elegida, como Dropout o BatchNormalization, dentro de la función build_mlp().

```

Python

def build_mlp():
    model = models.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(10, activation='softmax')
    ])
    return model, "MLP (Dense)"
  
```

Ejecutar

- Resultados obtenidos.

```

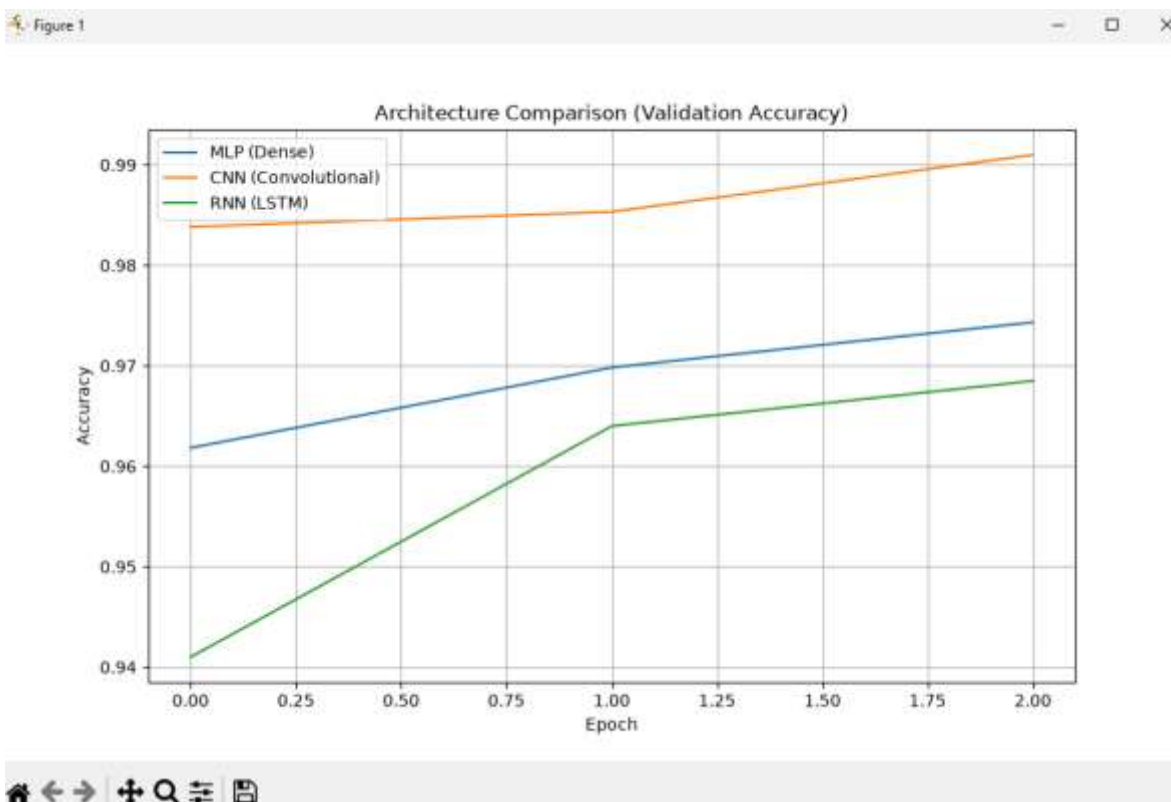
--- Training: MLP (Dense) ---
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CU
DA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
844/844 ██████████ 11s 4ms/step - accuracy: 0.8882 - loss: 0.3857 - val_accuracy: 0.9618 - val_loss:
0.1512
Epoch 2/3
844/844 ██████████ 3s 4ms/step - accuracy: 0.9419 - loss: 0.1971 - val_accuracy: 0.9698 - val_loss: 0
.1060
Epoch 3/3
844/844 ██████████ 3s 4ms/step - accuracy: 0.9571 - loss: 0.1493 - val_accuracy: 0.9743 - val_loss: 0
.0891
Test Accuracy (MLP (Dense)): 96.99%
  
```

```

--- Training: CNN (Convolutional) ---
Epoch 1/3
844/844 ██████████ 19s 19ms/step - accuracy: 0.9538 - loss: 0.1555 - val_accuracy: 0.9838 - val_loss:
0.0555
Epoch 2/3
844/844 ██████████ 16s 19ms/step - accuracy: 0.9861 - loss: 0.0451 - val_accuracy: 0.9853 - val_loss:
0.0492
Epoch 3/3
844/844 ██████████ 16s 19ms/step - accuracy: 0.9905 - loss: 0.0302 - val_accuracy: 0.9910 - val_loss:
0.0350
Test Accuracy (CNN (Convolutional)): 99.03%
  
```

```

--- Training: RNN (LSTM) ---
Epoch 1/3
844/844 ----- 14s 13ms/step - accuracy: 0.8149 - loss: 0.5711 - val_accuracy: 0.9410 - val_loss: 0.2015
Epoch 2/3
844/844 ----- 11s 13ms/step - accuracy: 0.9482 - loss: 0.1743 - val_accuracy: 0.9640 - val_loss: 0.1302
Epoch 3/3
844/844 ----- 11s 13ms/step - accuracy: 0.9636 - loss: 0.1200 - val_accuracy: 0.9685 - val_loss: 0.1038
Test Accuracy (RNN (LSTM)): 96.90%
    
```



- Resultados 2 prueba.

Evaluated Model	Original Accuracy (%)	Improved Accuracy (Regularization - Part 2)	Successfully Identified Tricky Drawing? (Yes / No)
MLP (Dense Fully-Connected)	96.93%	96.99%	[Insert Yes / No]
CNN (Spatial Convolutional)	98.87%	99.03%	[Insert Yes / No]
RNN (Sequential LSTM)	96.74%	96.90%	[Insert Yes / No]

- Resultados obtenidos test_my_handwriting.

```

--- Training: MLP (Dense) ---
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
844/844 ██████████ 10s 4ms/step - accuracy: 0.9002 - loss: 0.3493 - val_accuracy: 0.9598 - val_loss: 0.1449
Epoch 2/3
844/844 ██████████ 2s 3ms/step - accuracy: 0.9507 - loss: 0.1683 - val_accuracy: 0.9732 - val_loss: 0.0993
Epoch 3/3
844/844 ██████████ 2s 3ms/step - accuracy: 0.9624 - loss: 0.1262 - val_accuracy: 0.9742 - val_loss: 0.0869
Test Accuracy (MLP (Dense)): 97.32%

```

```

--- Training: CNN (Convolutional) ---
Epoch 1/3
844/844 ██████████ 15s 17ms/step - accuracy: 0.9533 - loss: 0.1549 - val_accuracy: 0.9830 - val_loss: 0.0537
Epoch 2/3
844/844 ██████████ 16s 19ms/step - accuracy: 0.9853 - loss: 0.0483 - val_accuracy: 0.9882 - val_loss: 0.0372
Epoch 3/3
844/844 ██████████ 22s 21ms/step - accuracy: 0.9901 - loss: 0.0313 - val_accuracy: 0.9895 - val_loss: 0.0350
Test Accuracy (CNN (Convolutional)): 98.78%

```

```

--- Training: RNN (LSTM) ---
Epoch 1/3
844/844 ██████████ 13s 13ms/step - accuracy: 0.8271 - loss: 0.5444 - val_accuracy: 0.9563 - val_loss: 0.1516
Epoch 2/3
844/844 ██████████ 11s 13ms/step - accuracy: 0.9522 - loss: 0.1622 - val_accuracy: 0.9682 - val_loss: 0.1094
Epoch 3/3
844/844 ██████████ 21s 14ms/step - accuracy: 0.9653 - loss: 0.1178 - val_accuracy: 0.9738 - val_loss: 0.0912
Test Accuracy (RNN (LSTM)): 96.98%

```

- Resultados con implementación de DROPOUT.

```

--- Training: MLP (Dense) ---
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
844/844 ██████████ 5s 5ms/step - accuracy: 0.8870 - loss: 0.3856 - val_accuracy: 0.9588 - val_loss: 0.1505
Epoch 2/3
844/844 ██████████ 3s 4ms/step - accuracy: 0.9419 - loss: 0.1963 - val_accuracy: 0.9698 - val_loss: 0.1112
Epoch 3/3
844/844 ██████████ 3s 3ms/step - accuracy: 0.9558 - loss: 0.1507 - val_accuracy: 0.9728 - val_loss: 0.0963
Test Accuracy (MLP (Dense)): 96.91%

```

```

--- Training: CNN (Convolutional) ---
Epoch 1/3
844/844 ██████████ 17s 18ms/step - accuracy: 0.9529 - loss: 0.1515 - val_accuracy: 0.9850 - val_loss: 0.0455
Epoch 2/3
844/844 ██████████ 16s 18ms/step - accuracy: 0.9860 - loss: 0.0454 - val_accuracy: 0.9877 - val_loss: 0.0457
Epoch 3/3
844/844 ██████████ 15s 18ms/step - accuracy: 0.9896 - loss: 0.0313 - val_accuracy: 0.9883 - val_loss: 0.0420
Test Accuracy (CNN (Convolutional)): 98.88%

```

```

--- Training: RNN (LSTM) ---
Epoch 1/3
844/844 ██████████ 11s 11ms/step - accuracy: 0.8169 - loss: 0.5686 - val_accuracy: 0.9453 - val_loss: 0.1809
Epoch 2/3
844/844 ██████████ 10s 12ms/step - accuracy: 0.9518 - loss: 0.1619 - val_accuracy: 0.9685 - val_loss: 0.1062
Epoch 3/3
844/844 ██████████ 11s 13ms/step - accuracy: 0.9667 - loss: 0.1128 - val_accuracy: 0.9737 - val_loss: 0.0938
Test Accuracy (RNN (LSTM)): 96.75%
(.venv) PS D:\NNA>

```

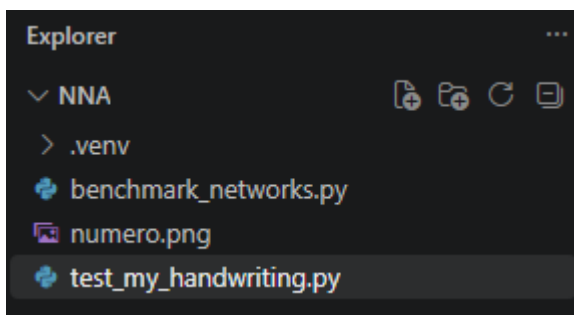
```

1/1 ————— 0s 142ms/step
1/1 ————— 0s 113ms/step
1/1 ————— 0s 181ms/step
MLP prediction: 3
CNN prediction: 1
RNN prediction: 7
(.venv) PS D:\NNA>

```

Evaluated Model	Original Accuracy (%)	Improved Accuracy (Regularization - Part 2)	Successfully Identified Tricky Drawing? (Yes / No)
MLP (Dense Fully-Connected)	97.32%	96.91%	NO
CNN (Spatial Convolutional)	98.78%	98.88%	NO
RNN (Sequential LSTM)	96.98%	96.75%	NO

- Archivos.



- Answers.

1. Internal Mechanics of the Adjustment:

The technique suggested by the AI was **Dropout**, in this case using something like `Dropout(0.3)` after the hidden Dense layer. Its purpose is to reduce overfitting by randomly deactivating a percentage of neurons during each training batch. Mathematically, this means that some neuron activations are multiplied by zero using a random binary mask. Because of that, the network cannot always depend on the same neurons or connections. The weight matrix itself is not permanently deleted or modified, but the effective path used during each batch changes. From a data engineering perspective, this helps the model generalize better because it reduces its tendency to memorize specific patterns from the training dataset.

2. Cognitive Analysis of the Stroke:

In the adversarial test made with Paint, the CNN would normally be the architecture that behaves most similarly to human visual reasoning, because it analyzes local spatial patterns such as edges, curves, and shapes while preserving the relationship between nearby pixels. This is closer to the way a person recognizes a handwritten number by its form instead of analyzing every pixel independently.

The MLP can produce the most counterintuitive error, especially when the digit is shifted, stretched, or drawn differently from the MNIST examples. Its input layer uses Flatten, converting the 28×28 image into a vector of 784 values. Mathematically, this destroys the explicit two-dimensional relationship between neighboring pixels. For example, pixels that form a curve are only interpreted as positions inside a long vector. Because of this, a small translation of the drawing can create a relatively large change in the input representation, even if a human still sees exactly the same number.

The RNN is somewhere between both approaches. It processes the image as 28 sequential rows, so it preserves some structure, but it is treating an image like a sequence, which is not necessarily the most natural representation for this problem.

3. Production Deployment Criteria:

For an automatic license plate recognition system operating in real time, I would choose a CNN. Even though an MLP can have relatively fast inference because its operations are mainly matrix multiplications, it is less robust when the position, angle, scale, or illumination of the characters changes.

A CNN is better suited to images because convolutional filters can detect useful local characteristics regardless of small changes in their exact position. This becomes important in an urban camera where plates can appear with rain, shadows, dirt, reflections, motion blur, or different distances from the camera.

The RNN/LSTM could be useful later for processing a sequence of detected characters, but I would not choose it as the main visual recognition architecture. Its sequential processing also introduces more computational dependency because each step depends on previous states, which can increase inference latency compared with highly parallel convolution operations.

Therefore, considering the balance between inference speed and robustness to real-world visual noise, the CNN offers the best option of the three. It may require more computation than a simple MLP, but that additional cost is justified because the model is much better adapted to spatial image information and to the variations that would occur in a real production environment.